UpMyRank

Complete Technical Documentation

AI-Powered JEE/NEET Tutoring Platform

Architecture · Codebase · Design Philosophy · Build Journey

April 14, 2026 · Version 1.0 · Internal Engineering Reference

Table of Contents

1. What Is UpMyRank?

UpMyRank is an AI-powered tutoring platform for Indian students preparing for JEE (engineering) and NEET (medicine) entrance exams. It covers Physics, Chemistry, and Maths at NCERT Class 11 and Class 12 level. The platform serves students preparing for the most competitive undergraduate entrance exams in India — approximately 1.5 million students attempt JEE each year.

Core Thesis

The problem with AI tutoring is not intelligence — it is pedagogy. Most AI tools simply give students the answer. UpMyRank refuses to do this. Instead, every response is generated through a Socratic engine that forces the student to think before receiving the next level of assistance. The system tracks what the student knows, how quickly they forget, and where their misconceptions lie — then adapts the teaching style accordingly.

Principle	What It Means	Why It Matters
Pedagogy-first AI	Every response deliberately withholds the answer until the student has attempted it	Desirable difficulty is intentional, not a bug
Adaptive Memory	Per-student Knowledge Genome tracks mastery via BKT, forgetting curves, and error fingerprints	Each student gets a unique path
Measurable Quality	4-dimension LLM judge evaluates every session against a golden dataset	Teach better, not just answer faster

2. The PTB Educational AI Framework

PTB (Python Tutor Bot) is the educational AI framework underlying UpMyRank. It provides a structured three-layer architecture that separates global rules from per-student adaptation and from quality measurement. The key insight is that the LLM is not the intelligence — the architecture is.

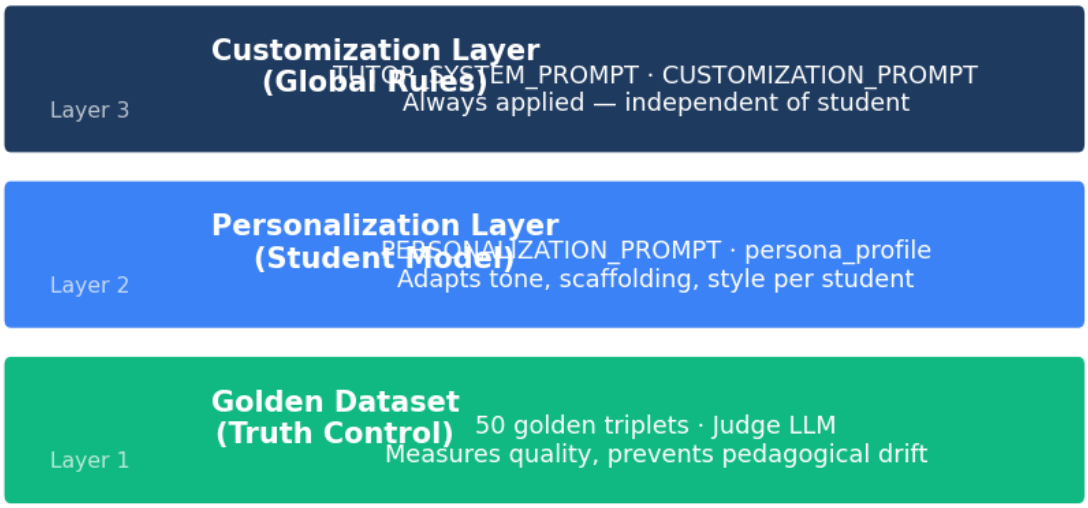


Figure: PTB three-layer architecture

Layer Details

Layer 1 — Customization (Global Rules): TUTOR_SYSTEM_PROMPT + CUSTOMIZATION_PROMPT. Defines what the AI always does, regardless of which student it is talking to. This is the global pedagogy contract: always Socratic, never give the answer directly, always cite NCERT.

Layer 2 — Personalization (Student Model): PERSONALIZATION_PROMPT + persona_profile. Adapts tone, scaffolding depth, preferred teaching style, and hint depth based on the individual student's onboarding profile and session history.

Layer 3 — Golden Dataset (Truth Control): 50 golden triplets + Judge LLM (gpt-4o-mini at temperature=0). Measures pedagogical quality on every session and fires pre-deploy regression gates. Prevents quality drift over time.

Core Philosophy

- The LLM is a composer, not the source of knowledge. The system architecture is the product.
- Optimize for learning gain, not user satisfaction. Desirable difficulty is intentional.
- Policy Engine first — always determine HOW to teach before generating a response.
- Misconceptions ≠ knowledge gaps. Wrong thinking requires different treatment than wrong answers.

• Measure pedagogy, not just accuracy. Judge LLM scores Socratic quality on every response.

3. The Build Journey — 10 Phases

UpMyRank was built in 10 sequential phases, each adding a distinct architectural layer. The sequence reflects a deliberate strategy: get the Socratic loop right first, then add memory, then add evaluation, then add personalization.

Phase	What Was Built	Key Files
Phase 0 Foundation	FastAPI app, asyncpg pool, Supabase Postgres, Docker/Redis, database schema	app/main.py app/db/database.py scripts/setup_db.sql
Phase 1 Socratic Engine	Hint ladder (levels 0–4), intent classification, forced-attempt gate, LaTeX sanitizer	app/services/doubt/engine.py app/services/doubt/prompts.py
Phase 2 Knowledge Genome	Per-concept EMA mastery tracking, doubt_blocks, session lifecycle	app/api/doubt.py app/api/session.py
Phase 3 Agentic RAG	4-tool agentic retriever, hybrid RRF search, pgvector HNSW index	app/services/rag/agent.py app/services/rag/tools.py app/services/rag/retriever.py
Phase 4 Student Memory	3-layer memory (Redis + Postgres + concept mastery), session summarizer	app/services/memory/context.py app/services/memory/summarizer.py
Phase 5 Policy Engine	scaffolding_level inference, PedagogyConfig, subject-specific policies	app/services/policy/engine.py
Phase 6 Misconception Detection	30-entry library, 1.5× mastery penalty, misconception presentation	app/services/doubt/misconceptions.py
Phase 7 Eval + Judge	4-dimension LLM judge, judge_evaluations table, regression gate, eval dashboard	app/services/eval/judge.py scripts/regression_gate.py
Phase 8 Onboarding + Persona	4-step UI onboarding, GPT-4.1-mini persona builder, persona evolution every 5 sessions	app/api/onboarding.py frontend/web/app/onboarding/page.tsx
Phase 9 Performance	Pre-computed embeddings, parallel tool dispatch, pg_trgm GIN indexes, concept seeding	app/services/rag/agent.py scripts/migrate_v14_perf_indexes.sql

4. Technology Stack

Every technology choice is driven by the educational AI requirements: async-first for streaming, vector similarity for RAG, EMA for mastery tracking, and no ORM to keep raw query control for complex pgvector operations.

Layer	Technology	Notes
Backend	FastAPI (Python 3.11), asyncpg, Pydantic v2	Async-first, no ORM
Primary LLM	gpt-4.1-mini	Socratic responses, hints, solutions, persona builder
Classification LLM	gpt-4o-mini	Intent classification, summarization, memory compression, judge
Vision LLM	gpt-4o	Image extraction only — never used for text generation
Vector DB	pgvector 0.8.2 on Postgres 16	HNSW index, 1536-dim embeddings, cosine distance
Embeddings	text-embedding-3-small	All tables uniform, 1536 dimensions
Cache	Redis (redis.asyncio)	Hot context 48h TTL, semantic cache cosine 0.92
Token counting	tiktoken cl100k_base	Prompt token budgets
Frontend	Next.js 14, TypeScript, Tailwind, Framer Motion	SSR + client components, SSE streaming
Package manager	Poetry (.venv in-project)	Python 3.11.14 from Miniconda
Config	pydantic-settings, .env	DATABASE_URL, OPENAI_API_KEY, REDIS_URL
Backend hosting	Render.com	Auto-deploy from main branch
Frontend hosting	Vercel	Edge CDN, Next.js native
DB hosting	Supabase (Postgres + RLS)	aws-0-us-west-2.pooler.supabase.com

5. Backend Architecture

Application Startup Sequence

FastAPI uses a lifespan context manager to initialize all shared services in a guaranteed order. Every service is stored on app.state so it is accessible to all route handlers without global singletons.

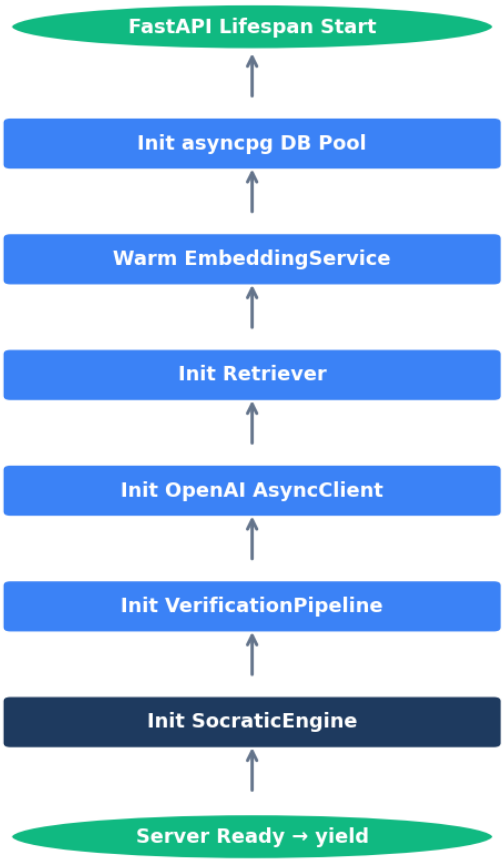


Figure: Lifespan startup — services initialized in dependency order

API Routers

Router	Prefix	Key Endpoints
health	/health	GET /
auth	/auth	POST /login, /signup, /refresh

onboarding	/onboarding	POST /submit, GET /status
admin	/admin	GET /metrics, GET /is_admin, GET /judge-metrics
doubt	/doubt	POST /ask/stream, POST /verify
feedback	/feedback	POST /response
session	/session	POST /start, /end, /resume
student	/student	GET /{id}
mock	/mock	POST /start, GET /submit
taxonomy	/taxonomy	GET /chapters

6. The Socratic Engine

The Socratic Engine is the heart of UpMyRank. Every student question passes through this pipeline, which decides what kind of response the student should receive — and deliberately withholds information until the student demonstrates effort.

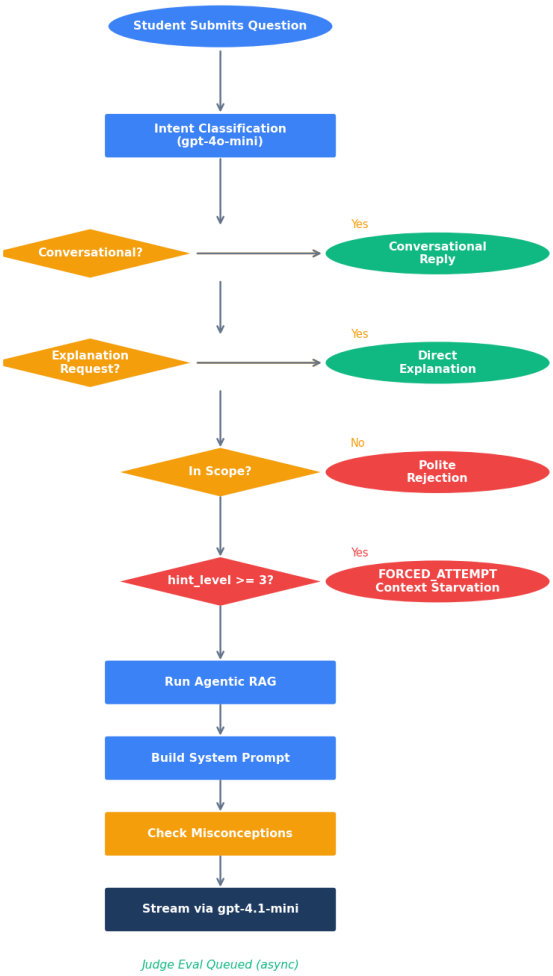


Figure: Full question-processing pipeline in the Socratic Engine

Intent Types

Intent	Trigger	Response Path
CONVERSATIONAL	"yes", "ok", "thanks", short affirmatives	CONVERSATIONAL_RESPONSE — no session started
EXPLANATION	"explain X", "what is X"	EXPLANATION_PROMPT — direct structured explanation, bypasses
OUT_OF_SCOPE	non-Physics/Chem/Maths topics	Polite rejection with subject scope reminder

SOCRATIC	First question on a concept	Hint Level 0: Socratic question only — no formula, no clue
CONTINUATION	Follow-up in existing session	get_hint() → escalate hint level based on student progress

7. The Hint Ladder

The hint ladder is the core of the Socratic method. It has 5 levels (0–4). The student starts at Level 0 and advances only when they demonstrate genuine effort. Level 3 is structurally enforced — it is not just a stricter instruction but a different system prompt with all RAG context removed.

Figure: Hint ladder — each level reveals more scaffolding

Level	Name	What the AI Does	What the Student Must Do
0	Socratic Question	Asks a probing question to activate prior knowledge. No formulas. No clues.	Think. Form a hypothesis. Write something.
1	Conceptual Bridge	Connects the question to a related concept the student already knows. Still no formula.	Show understanding of the bridge concept.
2	Formula/Method Guide	Reveals the relevant formula or method structure. **No substitution.**	Attempt substitution and derive the answer.
3	FORCED ATTEMPT	Context starved. Prompt swapped. AI: "You have everything — attempt now."	Write a full attempt. No escape hatch.
4	Full Solution	Step-by-step worked solution only after genuine attempt documented.	Review and compare against your attempt.

Level 3 Implementation Note: Level 3 is not enforced through instructions alone — the system prompt is completely swapped to SYSTEM_PROMPT_FORCED_ATTEMPT, all RAG context is cleared before the LLM call, and intent classification is skipped. This structural enforcement is necessary because LLMs can ignore instructional constraints under pressure. Context starvation makes it structurally impossible to give the answer.

8. Agentic RAG System

The Agentic RAG system uses an LLM-driven tool selection loop to retrieve the most relevant context for each question. Rather than always searching all sources, the agent decides which combination of tools to use based on the question and subject.

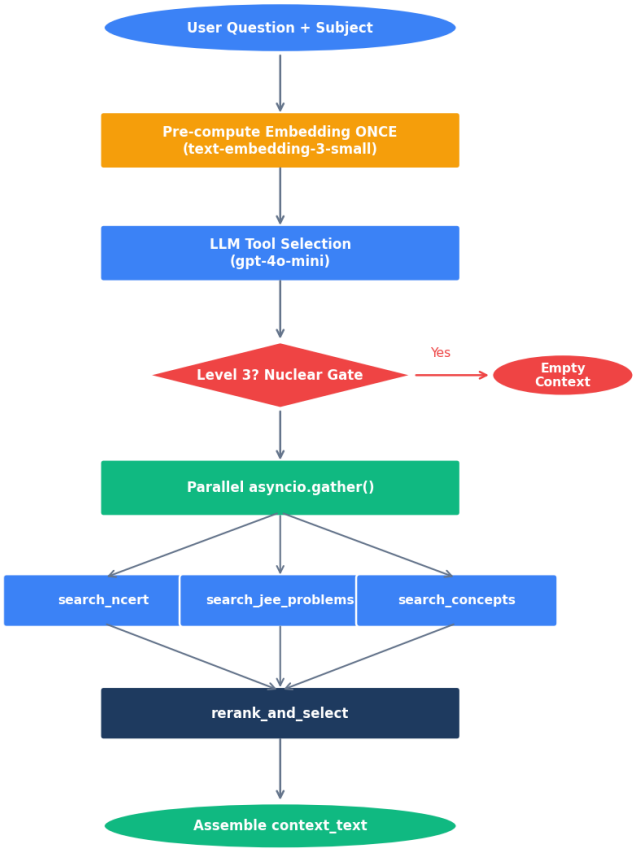


Figure: Agentic RAG retrieval pipeline

The 4 Tools

Tool	Purpose	Search Method
search_ncert	Find NCERT textbook passages relevant to the question	Hybrid RRF: pgvector cosine + pg_trgm ILIKE, fused with Reciprocal R
search_jee_problems	Find similar JEE Previous Year Questions	Vector similarity on 1536-dim embeddings, HNSW index
search_concepts	Look up concept definitions in the taxonomy	pg_trgm GIN index on subtopic/description

rerank_and_select	Select the best chunks from all retrieved results	LLM-guided reranking using precomputed embedding similarity

Hybrid RRF Formula

Each candidate gets: rrf_score = 1/(k + vector_rank) + 1/(k + text_rank) where k=60. Vector rank from pgvector cosine distance. Text rank from ILIKE pg_trgm GIN match. Final top-K selected by descending rrf_score. The RRF fusion approach means neither pure vector nor pure keyword dominates — exact formula name matches are boosted by text rank while semantic relevance is captured by vector rank.

Performance Optimizations

Optimization	Before	After	Improvement
Embedding calls per query	3 separate API calls (~300-700ms each)	1 precomputed, shared across all tools	~60-70% latency reduction
Tool dispatch	Sequential (tool1 → tool2 → tool3)	Parallel asyncio.gather()	~50% reduction for multi-tool step
ILIKE search	Full table scan on 14,384+ rows	pg_trgm GIN index	O(n) → O(log n)
Combined P95 latency	~12,677ms	Target <2,000ms	~84% improvement

9. Database Schema

The database runs on Postgres 16 hosted on Supabase with pgvector 0.8.2 for vector similarity search. All 14 tables have Row Level Security (RLS) enabled. Schema evolves through numbered migration files — never ad-hoc ALTER TABLE.

Table: students

One row per registered student. Source of truth for identity, onboarding state, and exam target.

Column	Type	Notes
id	UUID PK	Supabase auth.users FK
email	TEXT UNIQUE	Login identifier
name	TEXT	Display name
exam_type	TEXT	'JEE' or 'NEET'
class_level	TEXT	'11th', '12th', 'dropper'
physics_prev_marks	SMALLINT	0-100, from onboarding
study_hours_per_day	SMALLINT	From onboarding
exam_date	DATE	Target exam date
onboarding_completed	BOOLEAN	Gate for app access
created_at	TIMESTAMPTZ	Auto

Table: study_sessions

One row per study session. Contains the GPT-compressed summary and topics covered.

Column	Type	Notes
study_session_id	UUID PK	
student_id	UUID FK	→ students.id
started_at	TIMESTAMPTZ	Session start
ended_at	TIMESTAMPTZ	NULL if active
doubt_count	INT	Running count
session_summary	TEXT	GPT-4o-mini compressed summary
topics_covered	TEXT[]	All topics addressed this session

Table: doubt_blocks

One row per question asked. Tracks hint progression, misconception detection, and resolution.

Column	Type	Notes
doubt_block_id	UUID PK	
study_session_id	UUID FK	
doubt_session_id	UUID FK	→ doubt_sessions.id
topic	TEXT	Question topic string
subject	TEXT	'Physics', 'Chemistry', 'Maths'
hint_level	SMALLINT	0-4, current depth
solved	BOOLEAN	Student marked resolved
misconception_detected	BOOLEAN	From misconception library check
misconception_id	TEXT	Matched misconception key

Table: knowledge_chunks

15,069 NCERT passage chunks. Physics (10,505) + Chemistry (3,138) + Maths (1,426).

Column	Type	Notes
id	UUID PK	
content	TEXT	NCERT passage text (350-token chunks)
subject	TEXT	'Physics', 'Chemistry', 'Maths'
chapter	TEXT	Chapter name
class_level	TEXT	'11' or '12'
chunk_index	INT	Position within chapter
embedding	vector(1536)	text-embedding-3-small

Table: concept_mastery

Per-student per-concept mastery. The Knowledge Genome. Sole writer: _genome_update_task.

Column	Type	Notes
student_id	UUID FK	

concept_id	TEXT FK	→ concepts.id
mastery_score	FLOAT	EMA, 0.0–1.0
error_fingerprint	JSONB	{error_type: strength 0.0–1.0}
forgetting_rate	FLOAT	Ebbinghaus decay, 0.1–0.9
last_practiced	TIMESTAMPTZ	
PRIMARY KEY	(student_id, concept_id)	

Table: student_memory

Rolling compressed profile + full persona JSON. Evolves every 5 sessions.

Column	Type	Notes
student_id	UUID PK FK	
compressed_profile	TEXT	GPT-4o-mini rolling profile, ≤120 words
persona_profile	JSONB	Full persona dict (scaffolding_level, style, etc.)
persona_profile_updated_at	TIMESTAMPTZ	Staleness tracking
forgetting_rates	JSONB	{concept_id: rate} mirror for fast reads
sessions_since_compress	INT	Trigger: compress every 5 sessions

Table: session_events

Per-turn telemetry. Captures scaffolding scores, retrieval quality, and latency.

Column	Type	Notes
id	UUID PK	
doubt_block_id	UUID FK	
event_type	TEXT	'hint_requested', 'solved', etc.
scaffolding_score	SMALLINT	0-2 from Judge LLM
retrieval_similarity	FLOAT	Top chunk cosine similarity
response_latency_ms	INT	End-to-end response time
hint_was_useful	BOOLEAN	Feedback signal

Table: judge_evaluations

4-dimension judge scores. Populated async after session end.

Column	Type	Notes
id	UUID PK	
pedagogical_score	SMALLINT	0-2 (weight 0.40)
factual_score	SMALLINT	0-1 (weight 0.30)
context_relevance_score	SMALLINT	0-1 (weight 0.15)
hint_appropriateness_score	SMALLINT	0-1 (weight 0.15)
overall_score	FLOAT	Weighted composite 0.0–1.0
rationale_json	JSONB	Per-dimension rationale text

Table: jee_problems

20 verified JEE PYQs with embeddings for similarity search.

Column	Type	Notes
id	UUID PK	
question	TEXT	Problem text
answer	TEXT	Correct answer
subject	TEXT	Subject tag
year	INT	JEE year
difficulty	TEXT	easy/medium/hard
embedding	vector(1536)	For similarity search via HNSW

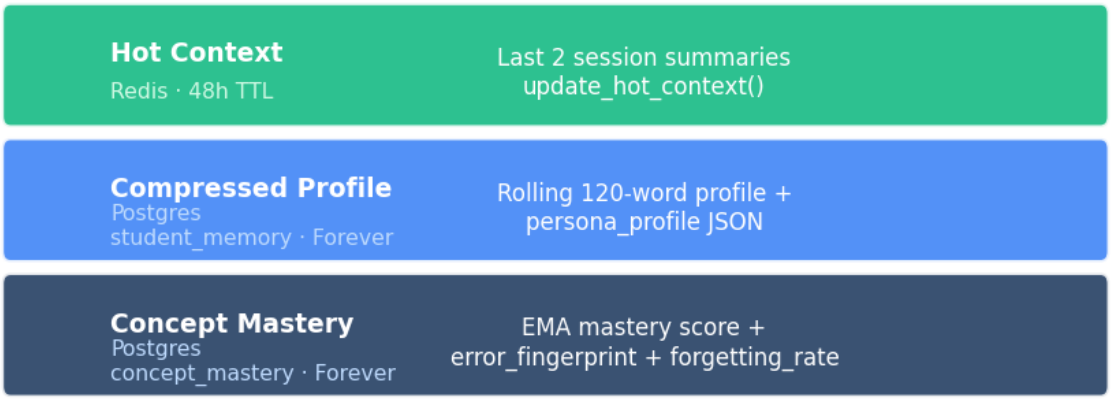
Migration Timeline

Migration File	What It Added
setup_db.sql	Base schema: students, study_sessions, doubt_blocks, doubt_sessions, knowledge_chunk
migrate_v4_memory.sql	student_memory table (compressed_profile, persona_profile, forgetting_rates)
migrate_v5_persona.sql	persona_profile JSONB column on student_memory
migrate_v6_misconceptions.sql	misconception_detected, misconception_id on doubt_blocks + session_events
migrate_v7_eval.sql	scaffolding_score, retrieval_similarity, response_latency_ms, hint_was_useful on session_

migrate_v8_onboarding.sql	onboarding_completed, class_level, physics_prev_marks, study_hours_per_day, exam_da
migrate_v9_persona_staleness.sql	persona_profile_updated_at on student_memory
migrate_v10_rls.sql	RLS enabled on all 10 public tables, 10 policies
migrate_v11_jee_problems.sql	jee_problems table, HNSW index, match_jee_problems() RPC
migrate_v12_feedback.sql	response_feedback, judge_evaluations, session_metrics tables
migrate_v13_subjects.sql	chemistry_prev_marks, maths_prev_marks, priority_subject, learning_preference on stude
migrate_v14_perf_indexes.sql	pg_trgm GIN indexes on content/subtopic, btree indexes on subject/chunk_index

10. 3-Layer Student Memory

The student memory system uses three layers with different speeds and lifetimes. Together they give the AI full context without overwhelming token budgets: hot recent summaries from Redis, compressed profile from Postgres, and per-concept mastery scores.



Layers stack: each session reads all three, writes to appropriate layer

Figure: 3-layer memory architecture

Layer	Storage	TTL	Written By	Read By
Hot Context	Redis	48 hours	update_hot_context() on session end	build_context_bundle()
Compressed Profile	Postgres student_memory	Forever	maybe_compress_profile() every 5 sessions	build_context_bundle()
Concept Mastery	Postgres concept_mastery	Forever	_genome_update_task in doubt.py (sole writer)	build_context_bundle() → top 5 weakest concepts

persona_profile JSON Structure

The persona_profile is a JSONB column on student_memory. It is built during onboarding and partially rewritten every 5 sessions by maybe_compress_profile(). Structured keys are preserved via dict merge — only persona_summary is ever fully overwritten.

```
{
  "scaffolding_level": "HIGH | MEDIUM | LOW",
  "preferred_style": "analogy | formula | example | visual",
  "common_misconceptions": ["string", ...],
  "allowed_hint_depth": 3,
  "interaction_depth_score": 0.0,
  "learning_velocity": 0.0,
  "persona_summary": "3-4 sentence student description"
}
```

Persona Evolution (Every 5 Sessions)

• Onboarding builds the initial persona_profile from survey answers via GPT-4.1-mini
• Every 5 sessions: maybe_compress_profile() fires as background task from /session/end
• Uses last 10 session summaries + top 10 concept mastery scores as input
• Two GPT-4o-mini calls: (1) update compressed_profile paragraph, (2) rewrite persona_summary
• Preserves all structured keys (scaffolding_level, preferred_style, etc.) via dict merge — never fully overwrites
• persona_profile_updated_at tracks freshness; format_context_for_prompt() adds staleness warning if >15 sessions old

11. Policy Engine

The Policy Engine is a separate architectural layer that runs before every LLM call. It produces a PedagogyConfig dataclass that determines HOW to teach — independent of WHAT to teach. Separating these concerns is the key architectural decision that makes pedagogy deterministic and auditable.

Scaffolding Level Inference

avg_mastery Range	scaffolding_level	Teaching Style
< 0.4	HIGH	Explicit, step-by-step, analogies, check-ins after each concept
0.4 – 0.7	MEDIUM	Formula-focused, neutral tone, moderate pacing
> 0.7	LOW	Application problems, high density, direct tone, minimal scaffolding

Subject-Specific Overrides

Subject	HIGH	MEDIUM	LOW	Special Rules
Physics	conceptual	formula	application	None
Chemistry	conceptual	formula	application	HIGH: use_analogies=False (equations are the intuition vehicle)
Maths	conceptual	application	application	max_concepts -= 1 (each step needs verification)

hint_level Overrides

• hint_level == 0: always socratic_style = "conceptual" — regardless of scaffolding_level
• hint_level >= 3: check_in_required = False — forced attempt means no mid-response pacing

12. Misconception Detection

Misconceptions are qualitatively different from knowledge gaps. A student who has never learned a concept needs teaching. A student with an active misconception needs correction — often counter-intuitive correction that directly confronts the wrong model. UpMyRank maintains a 30-entry library and applies a 1.5× mastery penalty when a match is found.

Field	Value
Library size	30 entries covering Physics, Chemistry, and Maths
Check function	check_for_misconception(response, topic, subject) → Misconception|None
Penalty	1.5× mastery penalty applied to affected concept via EMA
Storage	misconception_detected=TRUE, misconception_id stored on doubt_block
Trigger	Runs on every AI response before returning to the student

Example Misconceptions

Subject	Misconception	Correct Model
Physics (Mechanics)	Heavier objects fall faster under gravity	All objects fall at the same rate in absence of air resistance (Galileo)
Physics (Electricity)	Current is "used up" as it flows through a circuit	Charge is conserved; voltage drops across components, not current
Chemistry (Acids)	Acids always contain oxygen	HCl, HBr, HF contain no oxygen — acidity is defined by H+ donation
Chemistry (Bonds)	Ionic bonds are stronger than covalent bonds	Depends on the specific compounds; many covalent bonds are stronger
Maths (Calculus)	Derivatives only measure the slope of straight lines	Derivatives measure instantaneous rate of change — applicable to any c
Maths (Probability)	P(A and B) = P(A) × P(B) always	This only holds when A and B are independent; joint probability requires

13. Judge Evaluation System

Every session is scored by a 4-dimension LLM judge (gpt-4o-mini at temperature=0). The judge fires asynchronously after session end via asyncio.create_task() — it never blocks the user-facing response. Results power the admin dashboard and the pre-deploy regression gate.

4-Dimension Scoring Formula

Dimension	Scale	Weight	What It Measures
Pedagogical	0-2	0.40 (40%)	Socratic quality — did AI withhold answer appropriately?
Factual	0-1	0.30 (30%)	Factual accuracy of the response content
Context Relevance	0-1	0.15 (15%)	Did AI use the RAG context that was provided?
Hint Appropriateness	0-1	0.15 (15%)	Right level of scaffolding for student's current state?
Overall (composite)	0.0–1.0	—	0.4×(ped/2) + 0.3×fact + 0.15×ctx + 0.15×hint

Judge Pipeline

- Judge fires async from POST /session/end via asyncio.create_task() — never blocks user response
- Uses gpt-4o-mini at temperature=0 for deterministic, reproducible scoring
- Input: question (truncated to 4000 chars) + AI response (truncated to 4000 chars)
- Output: {pedagogical_score, factual_score, context_relevance_score, hint_appropriateness_score, overall_score, rationale_json}
- Results stored in judge_evaluations table with FK to both study_session and doubt_session
- Admin dashboard aggregates: adherence rate, avg scores, per-topic breakdown, drifting topics (score < 1.5)
- Regression gate (scripts/regression_gate.py): exit code 1 if overall_score < 0.6 on golden dataset — blocks deploy
- Pedagogy drift report (scripts/pedagogy_drift_report.py): weekly, flags topics with avg score < 1.5

Admin Dashboard Metrics

Metric	Endpoint	Description
Socratic Adherence Rate	GET /admin/metrics	Fraction of sessions scoring pedagogical ≥ 1 (out of 2)
Avg Retrieval Similarity	GET /admin/metrics	Mean cosine similarity of top chunk across all sessions
Latency P95	GET /admin/metrics	95th percentile response latency in milliseconds
Per-Topic Breakdown	GET /admin/metrics	Average overall score per topic — sorted ascending to surface weak topics
Judge Score Trend	GET /admin/judge-metrics	Time series of overall_score over last N days

14. Onboarding Flow

New students complete a 4-step onboarding flow before accessing the platform. This is the moment the system's first impressions of the student are formed — the resulting persona_profile drives all subsequent pedagogy decisions until the session evidence overrides it.

Step	What's Collected	How It's Used
1: Academic Background	class_level, physics_prev_marks, chemistry_prev_marks, maths_prev_marks	Initial scaffolding_level inference; weakest subject
2: Topic Assessment	easy_topics[], hard_topics[] — Physics (16 topics), Chemistry (16), Maths (16) = 48 total	Seeds mastery_scores (easy=0.7, ha
3: Study Plan	study_hours_per_day, exam_type, exam_date, priority_subject	Study intensity + subject weighting; learning_style (formula/analogy/example/vis
4: Persona Summary	— (display only)	Shows generated persona card (3-4 sentences describing student mod

Persona Builder

• GPT-4.1-mini processes all survey answers and produces the persona_profile JSON

• persona_summary: 3-4 sentences covering ability level per subject, hint preference, emotional pattern, teaching style

• scaffolding_level derived from weakest-subject marks (lowest of physics/chemistry/maths)

• preferred_style: set directly from learning_preference input — explicit student input is authoritative

• common_misconceptions: seeded from hard_topics[] based on subject misconception library

• allowed_hint_depth: defaults to 3 for first-time students; policy engine can override

15. Frontend Architecture

The frontend is a Next.js 14 application with TypeScript, Tailwind CSS, and Framer Motion. It uses no Redux — state is managed with local useState and context. API calls use a custom fetchWithRetry() with exponential backoff and automatic token refresh on 401 errors.

Pages

Page	Route	Purpose
Home	/	Dashboard: persona greeting, 3 subject mastery cards, exam countdown, continue sessi
Auth Login	/auth/login	Supabase auth + onboarding status check → redirects to /onboarding if not done
Auth Signup	/auth/signup	Always redirects new signups to /onboarding
Onboarding	/onboarding	4-step glassmorphic flow: marks → topics → study plan → persona summary card
Doubt	/doubt	Main chat UI with SSE streaming, topic lock, subject badge, confidence meter
Practice	/practice	Topic-based practice (planned)
Mock	/mock	Full mock exam simulation with timer and MCQ interface
Progress	/progress	Knowledge Genome mastery visualization per concept
Settings	/settings	4-tab settings: Profile / My Analytics / System Analytics / Preferences
Admin	/admin	System eval dashboard — admin-gated via ADMIN_STUDENT_ID env var

Key Components

Component	Purpose
ChatMessage.tsx	Renders AI/student messages with LaTeX via KaTeX, badge display, thumbs up/down feedback butt
ChatInput.tsx	Input + ConfidenceMeter with AnimatePresence swap; base64 image upload via FileReader (no Supa
Sidebar.tsx	280px desktop nav; student identity card (avatar + name + exam); TopicTree; Learning Profile section
TopicTree.tsx	Subject tabs (Phy/Che/Mat); chapter accordion with mastery bar; topic row with Doubt/Practice/Mock
QuickDoubtFAB.tsx	56px FAB globally mounted; bottom-sheet textarea; navigates to /doubt?q=question; hidden on /doub

SSE Streaming

The doubt page uses EventSource-equivalent fetch with ReadableStream. Every token is yielded as {"token": "...", "done": false}. The final event is {"done": true} which closes the stream. A keepalive {"done": false, "thinking": true} is sent immediately on continuation requests to prevent Render's 30-second proxy timeout from

killing the connection before the LLM call completes.

API Client: fetchWithRetry()

• Retry delays: 5s / 15s / 30s (covers Render cold start of ~20s)
• Automatic 401 handling: calls tryRefresh() to get new access token, retries original request
• If refresh fails: redirect to /auth/login
• pingBackend(): called on mount of login/signup/onboarding pages to warm Render cold start

16. Hard Invariants — The Rules

These 9 rules are hard invariants. Violating them creates silent bugs — the kind that corrupt data over time without throwing visible errors. They must never be broken.

#	Rule	Why It Matters
1	_genome_update_task is the sole mastery writer	Any second EMA path creates split-brain mastery — student model becomes inconsistent a
2	summarize_session() is always awaited — never fire-and-forget	Fire-and-forget breaks the blocking requirement; session must end with summary written
3	Redis errors must never propagate	Cache is optional infrastructure — a Redis crash must never take down user-facing flows
4	Level 3 = context starvation, not just instruction	LLMs ignore instructions under pressure. Structural removal is the only reliable enforc
5	gpt-4o is for vision only, never text	Cost and latency reasons; gpt-4.1-mini delivers equivalent Socratic quality for text genera
6	LaTeX sanitizer runs on every LLM response	Malformed LaTeX breaks the frontend KaTeX renderer for ALL users — never send raw
7	DB migrations are files in scripts/	Ad-hoc ALTER TABLE creates irreproducible schema state across environments — alwa
8	Admin gate is ADMIN_STUDENT_ID env var, not Supabase RLS	No Supabase role changes needed; simple, auditable, easy to revoke
9	Persona evolution preserves structured keys	Only persona_summary is rewritten; scaffolding_level and other keys survive via dict mer

17. Architecture Decisions

Every major architectural decision was driven by pedagogical requirements, not convenience. This section documents what was chosen, what was rejected, and why.

Decision	Chosen	Rejected	Reason
Retrieval strategy	Agentic RAG (LLM tool selects search)	Single-shot vector search	Static retrieval misses subject context; agent routes correctly be
Search fusion	Hybrid RRF (pgvector + pg_trgm, k=60)	Pure vector, pure keyword	Vector alone misses exact formula names; keyword alone miss
Mastery tracking	EMA per concept (Exponential Moving Average)	Binary correct/incorrect, Simple average	EMA captures learning trajectory and forgetting — not just the l
Error fingerprint	JSONB {error_type: 0.0–1.0} with decay	Simple error list	Strength-based model allows targeted remediation and tracks w
Pedagogy layer	Separate Policy Engine, runs before LLM	Inline prompt instructions	Separation allows deterministic pedagogy decisions that can be
Level 3 enforcement	Structural context removal	Stronger instructions	Instructions alone cannot reliably prevent LLM from giving the a
Session summarizer	await summarize_session() (blocking)	asyncio.create_task()	Summary must be written before session ends for next-session
Persona evolution	maybe_compress_profile() every 5 sessions	Per-session update	Real-time GPT call per session too expensive; 5-session batchi
Database auth	Supabase RLS on all 10 tables	Backend-only auth	Defense in depth; student_id from auth.uid() prevents horizonta
Embedding strategy	Single embed_single() at start of agentic loop	Embed per tool call	Eliminates 2 of 3 OpenAI API calls per retrieval; ~60% latency r
Image handling	base64 via FileReader.readAsDataURL()	Supabase Storage upload	No Supabase env vars needed on Vercel; simpler, no storage c

18. End-to-End Data Flow

The complete journey of a single student question — from keypress to genome update. This is the critical path through every layer of the system.

Figure: Complete end-to-end data flow for a student question

Step-by-Step Walkthrough

1. Input: Student types question in doubt/page.tsx ChatInput. Subject and topic may be pre-set from TopicTree navigation.

2. API Call: POST /doubt/ask/stream fires. Body: {question, subject, study_session_id, doubt_session_id, hint_level}.

3. Intent: gpt-4o-mini classifies as SOCRATIC (or CONTINUATION, EXPLANATION, CONVERSATIONAL, OUT_OF_SCOPE).

4. RAG: AgenticRetriever.run(): embed once → gpt-4o-mini tool selection → parallel asyncio.gather() → hybrid RRF → top chunks.

5. Context: build_context_bundle() pulls: Redis hot context (last 2 summaries) + Postgres compressed_profile + top 5 weakest concepts.

6. Policy: select_pedagogy(persona_profile, topic, hint_level) returns PedagogyConfig: scaffolding_level, socratic_style, max_concepts, check_in_required.

7. Misconception: check_for_misconception(response, topic, subject) checks against 30-entry library. Match → 1.5× penalty flag.

8. Prompt: build_system_prompt() assembles full prompt: TUTOR_SYSTEM_PROMPT + CUSTOMIZATION_PROMPT + PERSONALIZATION_PROMPT + student context block.

9. Streaming: gpt-4.1-mini streams response token-by-token as SSE events {"token": "...", "done": false}.

10. Sanitize: _sanitize_latex() runs on each chunk before sending to client. Frontend renders with KaTeX.

11. Resolve: Student clicks "Got it" → POST /doubt/hint with student_resolved=true → _genome_update_task fires EMA mastery update.

12. Session End: POST /session/end → await summarize_session() (blocking) → update_hot_context() (Redis) → asyncio.create_task(maybe_compress_profile()) (background) → asyncio.create_task(judge_eval) (background).

Knowledge Base Summary

Metric	Value
Total knowledge chunks	15,069
Physics chunks	10,505 (Class 11 + 12, all chapters)
Chemistry chunks	3,138 (Class 11 + 12, ingested from KadamParth HuggingFace dataset)
Maths chunks	1,426 (Class 11 + 12, ingested from NCERT PDFs via pdfplumber)
JEE PYQs	20 verified seed problems (Physics + Chemistry + Maths)
Total concept nodes	199 (Physics: 84, Chemistry: 62, Maths: 53)
Embedding model	text-embedding-3-small, 1536 dimensions, uniform across all tables
Embedding index	HNSW (pgvector 0.8.2), cosine distance metric
Text search index	pg_trgm GIN on content + subtopic columns
Chunk size	~350 tokens with 50-token overlap